

CS 253 Project 1 Mark Holliday

README_PODMAN

Podman is a widely-used containerization platform that has several advantages over Docker. Below we describe how to use Podman in this project.

Installation

WSL2

In a WSL2 Ubuntu terminal.

```
sudo apt update
sudo apt install -y podman
```

Verify

```
podman --version
```

macOS

Install using Homebrew

```
brew install podman
```

Initialize the Podman machine (required on macOS, once)

```
podman machine init
```

Start the machine

```
podman machine start
```

Verify

```
podman --version
```

Why the **machine**? Containers are a Linux feature, so on macOS Podman runs them inside a small Linux virtual machine. The machine must be running before any other **podman** command will work.

Linux (Ubuntu/Debian)

Update package list

```
sudo apt update
```

Install Podman

```
sudo apt install -y podman
```

Verify installation

```
podman --version
```

Output: `podman version 3.4.4` or `podman version 4.9.3`, depending on your Ubuntu release. Any 3.4 or later version works for this project.

The files you were given

File	Purpose
<code>Containerfile.network</code>	Recipe for the image of the <code>network</code> service
<code>Containerfile.route</code>	Recipe for the image of the <code>route</code> service
<code>Containerfile.routeSummary</code>	Recipe for the image of the <code>routeSummary</code> service
<code>Containerfile.bestJourneys</code>	Recipe for the <code>bestJourneys</code> service (Project 2)
<code>Containerfile.disruptions</code>	Recipe for the <code>disruptions</code> service (Project 2)
<code>create_and_run_containers.sh</code>	Builds the images, creates the pod, starts the containers
<code>delete_containers.sh</code>	Stops and removes the pod, then removes the images
<code>.containerignore</code>	Lists files that are kept <i>out</i> of the build

Copy all of these into the directory that already holds your service source files. They are meant to sit beside your code, not in a subdirectory of their own.

You do not use the `Containerfile.bestJourneys` and the `Containerfile.disruptions` files in Project 1; you will use them in Project 2.

Before you build

1. Everything lives in one directory

Each `podman build` command in the script ends with a `.`, which sets the **build context**: the directory Podman is allowed to copy files from. Your sources, your `package.json`, and your `.json` data files must all be in that one directory, alongside the Containerfiles.

```

Your project directory (.)      ← this is the build context
├─ Containerfile.network
├─ Containerfile.route
├─ Containerfile.routeSummary
├─ Containerfile.bestJourneys
├─ Containerfile.disruptions
├─ .containerignore
├─ create_and_run_containers.sh
├─ delete_containers.sh
├─ package.json                ← COPY package*.json ./ reads this
├─ railway_network.js          ← COPY . . reads these
├─ railway_route.js
├─ railway_routeSummary.js
├─ railway_bestJourneys.js     ← Project 2
├─ railway_disruptions.js      ← Project 2
├─ client_network.js           ← the clients stay on the host; they are
├─ client_route.js             copied into the image but never run there
├─ client_routeSummary.js
├─ singleton.json              ← the railway data files
├─ smokey.json
└─ uk.json

```

A file outside this directory cannot be copied into the image. A file in a *subdirectory* of it is copied into the matching subdirectory of the image, where `CMD ["node", "railway_network.js"]` will not find it. Keep the service files flat.

2. `package.json` must list the dependencies

Each Containerfile installs your dependencies with

```

COPY package*.json ./
RUN npm install --omit=dev

```

That reads the `dependencies` section of `package.json` and installs nothing else. If that section is missing or empty, the image is built without `express` and `axios`, and the container dies immediately with `Error: Cannot find module 'express'`.

So before your first build, from your project directory on the host, run

```
npm install axios express
```

This records both packages in `package.json`. Confirm it worked:

```
cat package.json
```

You should now see something like

```
"dependencies": {
  "axios": "^1.13.2",
  "express": "^5.1.0"
}
```

3. `.containerignore`

```
node_modules
coverage
.git
*.md
```

These patterns are skipped by `COPY . ..`. Keeping `node_modules` out matters most: without it your host's copy would overwrite the one `npm install` just built inside the image, and the two are not interchangeable. A Mac's `node_modules` contains macOS binaries, which will not run on the container's Linux.

Ports

This is the part students most often get wrong, so it is worth a table.

Service	Source file	Port inside pod	Port on your machine	Image name	Container name
network	<code>railway_network.js</code>	3000	30600	<code>railway_network</code>	<code>network</code>
routeSummary	<code>railway_routeSummary.js</code>	3001	30601	<code>railway_routesummary</code>	<code>routesummary</code>
route	<code>railway_route.js</code>	3002	30602	<code>railway_route</code>	<code>route</code>
bestJourneys	<code>railway_bestJourneys.js</code>	3003	30603	<code>railway_bestjourneys</code>	<code>bestjourneys</code>
disruptions	<code>railway_disruptions.js</code>	3004	30604	<code>railway_disruptions</code>	<code>disruptions</code>

The last two rows are yours to add in Project 2.

There are two sets of numbers because there are two sides of a boundary:

- **Inside the pod**, your server code calls `app.listen(3000)`, and the services reach each other at `http://localhost:3000`, `http://localhost:3001`, and so on. Containers in one pod share a network, so `localhost` inside any of them means the pod.
- **On your machine**, those ports are reached at 30600, 30601, and so on, because `podman pod create` maps them. Your client programs run on the host, so they use the 306xx numbers.

Note also that image names are lowercase: `railway_routesummary`, not `railway_routeSummary`. Podman requires lowercase image names, so the script already uses them.

`create_and_run_containers.sh`

Run the shell script

From your project directory do the following.

```
./create_and_run_containers.sh
```

If you get `Permission denied`, the executable bit is not set. Either fix it once

```
chmod +x create_and_run_containers.sh delete_containers.sh
```

or invoke the interpreter directly, which does not require it

```
bash create_and_run_containers.sh
```

The script begins with `set -e`, so it stops at the first command that fails rather than continuing and producing a half-built pod.

macOS

On macOS you must start the Podman virtual machine first. The line

```
# podman machine start
```

near the top of the script is commented out. Uncomment it. On WSL2 and on native Linux, leave it commented out.

Build the container images

The first part of the script builds one image per service.

```
podman build -f Containerfile.network      --tag railway_network      .
podman build -f Containerfile.routeSummary --tag railway_routesummary .
podman build -f Containerfile.route        --tag railway_route        .
```

- The `-f` gives the path to the Containerfile for this service (the same idea as a Dockerfile).
- The `--tag` gives the name for the image being built.
- The trailing `.` is the build context, the directory `COPY` reads from, as described above.

`podman build` then follows the steps in the named Containerfile to produce one image.

What the Containerfile does

The five Containerfiles are identical apart from the `EXPOSE` line and the file named in `CMD`. Reading `Containerfile.network`:

```
FROM node:22-bookworm-slim
```

Start from a published Node image. The major version is pinned deliberately. `node:latest` would change under you during the semester, and your build would stop matching the one used for grading.

```
WORKDIR /usr/src/app
```

Every later command runs in this directory inside the *container's* file system, not yours.

```
COPY package*.json ./
RUN npm install --omit=dev
```

The dependency list is copied and installed *before* the source code. Because this step does not mention your `.js` files, editing `railway_network.js` does not force `npm install` to run again on the next build. Podman reuses the cached layer and the rebuild is much faster.

```
COPY . .
```

Now the rest of the source and the `.json` data files, minus whatever `.containerignore` excludes.

```
USER node
```

Everything after this runs as the unprivileged `node` user rather than `root`.

```
EXPOSE 3000
```

Documentation only: it records which port this service listens on inside the pod. It does **not** publish anything. Publishing happens in `podman pod create`.

```
CMD ["node", "railway_network.js"]
```

The command that runs when a container is started from this image.

Create the pod

```
podman pod create --name railway \
  -p 30600:3000 \
  -p 30601:3001 \
  -p 30602:3002
```

`podman pod create` creates a pod: a group of containers that share one network (reachable at `localhost`), one IP address, and one set of ports.

- `--name railway` is the name of the pod.
- Each `-p` is a mapping written `host:container`. The left number is the port on your machine, which your client programs call; the right number is the port inside the pod, which your server code listens on.

Ports must be published here, on the pod, and not on `podman run`. The pod owns the network, so a `-p` on an individual container in a pod has no effect.

Run the containers

```
podman run -d --pod railway --name network      railway_network
podman run -d --pod railway --name routesummary railway_routesummary
podman run -d --pod railway --name route        railway_route
```

`podman run` starts a container from an image.

- `-d` runs it detached, in the background, so your command prompt returns.
- `--pod railway` puts this container in the pod named `railway`.
- `--name network` names the container, which is how you refer to it in `podman logs`, `podman stop`, and `podman start`.
- The last argument, `railway_network`, is the **image** to run.

The pod name and the image name are separate arguments. Confusing the two is the most common error here.

Check that it worked

The script ends by listing what is running.

```
podman ps --pod
```

You should see three containers, all with status `Up`, all in the pod `railway`.

A container that exited immediately has almost always failed at startup. Read its output:

```
podman logs network
```

`Cannot find module 'express'` means `package.json` had no dependencies section; see step 2 above. `EADDRINUSE` means two services are trying to listen on the same port inside the pod.

Use the pod

Once the pod is up you can use the services from the host. Your clients call the **host** ports, 30600 and up:

```
node client_network.js singleton.json
node client_routeSummary.js singleton.json
node client_route.js singleton.json Singleton Betaford Epsilon
```

The clients as provided have a line near the top like

```
const BASE_URL = 'http://localhost:3000';
```

That address is correct only when the servers are running natively on your machine. Once they run in the pod, each client must use the matching host port from the table above:

```
const BASE_URL = 'http://localhost:30600';
```

`client_routeSummary.js` becomes 30601 and `client_route.js` becomes 30602. The server files are not changed: inside the pod they still listen on 3000, 3001, and 3002, respectively.

delete_containers.sh

Run the shell script

Delete the pod and the images it was built from.

```
./delete_containers.sh
```

or

```
bash delete_containers.sh
```

Stop the pod

```
podman pod stop railway
```

Stopping the pod stops every container inside it, which ensures a clean shutdown of all services.

Remove the pod

```
podman pod rm railway
```

This removes the pod and its containers, but not the images they were built from, which still occupy disk space until removed below.

Remove the images

```
podman rmi localhost/railway_network
podman rmi localhost/railway_routesummary
podman rmi localhost/railway_route
```

- `podman rmi` removes an image.
- `localhost/railway_network` is the image's full name. When you build an image locally, Podman stores it under the registry name `localhost`, so that prefix is part of the name. It is not a file path.

The script finishes with `podman images` so you can confirm what is left.

Project 2: adding the two new services

`Containerfile.bestJourneys` and `Containerfile.disruptions` are already written for you. What you must do is add them to the two shell scripts. In `create_and_run_containers.sh` there are three edits, one in each numbered section:

- a `podman build` line for each new service;
- two more mappings on `podman pod create, -p 30603:3003` and `-p 30604:3004`;
- a `podman run` line for each new service.

In `delete_containers.sh`, add `podman rmi` lines for `localhost/railway_bestjourneys` and `localhost/railway_disruptions`.

Remember that no two containers in a pod may listen on the same port, since they share one network.

Troubleshooting

Symptom	Cause
Permission denied running ./create_and_run_containers.sh	Executable bit not set; use <code>chmod +x</code> or run with <code>bash</code>
Cannot find module 'express' in podman logs	<code>package.json</code> has no dependencies; run <code>npm install axios express</code> and rebuild
pod already exists	A previous pod is still there; run <code>./delete_containers.sh</code> first
EADDRINUSE inside a container	Two services listening on the same port inside the pod
Client hangs or gets ECONNREFUSED	Client is calling 3000 instead of the host port 30600
Everything fails on macOS	The Podman machine is not running; <code>podman machine start</code>